

Capítulo 2

Representación molecular y optimización geométrica

Ahora que tenemos una idea más clara sobre qué es un modelo y una simulación, o cómo podemos integrar comandos básicos de Python para resolver algunos problemas mediante una estrategia clara de pasos (algoritmos), o que diferenciamos MM, de HF y de DFT, procederemos a digitalizar a las moléculas, de esta manera podremos emplearlas para realizar cálculos.

La química es una ciencia maravillosa. En cada lugar de la naturaleza podemos encontrar un vasto ambiente para explorar y aprender de la mano de las teorías químicas. El aprendizaje clásicamente proviene de la experimentación y la construcción del conocimiento empírico, sin embargo, la ciencia ha avanzado hacia nuevos paradigmas, pasando por la ciencia teórica (mecánica, termodinámica, leyes de Maxwell, etcétera), la ciencia computacional o de las simulaciones (teoría del funcional de la densidad, dinámica molecular, etcétera) y la ciencia de los datos, y hago énfasis de la gran cantidad de datos (inteligencia artificial, minería de datos, reconocimiento de patrones, etcétera) [7].

La química digital es el campo que integra la ciencia química con tecnologías digitales, computación avanzada, ciencia de datos, inteligencia artificial, automatización y laboratorios conectados, para modelar, predecir, controlar y acelerar descubrimientos y procesos químicos. En vez de depender sólo de experimentos manuales, la química digital combina simulación teórica, aprendizaje automático, flujos de trabajo automatizados y gestión de datos para diseñar las moléculas, optimizar reacciones y se gestionan resultados experimentales [2].

La química computacional y la quimioinformática se han consolidado como pilares fundamentales de la investigación química moderna, permitiendo la predicción de propiedades moleculares, el diseño racional de fármacos y materiales, y la interpretación de fenómenos fisicoquímicos complejos. Sin embargo, el acceso efectivo a estas herramientas no está distribuido de manera equitativa a nivel global. Diversos estudios han señalado que las brechas en infraestructura computacional, licencias de software propietario y formación especializada limitan la participación de instituciones de países en desarrollo en investigación computacional de frontera [5]. Uno de los principales factores que contribuyen a estas brechas es la dependencia histórica de software propietario de alto costo y de infraestructura de cómputo de alto desempeño. Aunque los centros de investigación en países industrializados cuentan con acceso a clústeres y aceleradores especializados, muchas universidades carecen de los recursos financieros necesarios para sostener estas inversiones. En este contexto, el desarrollo y adopción de software libre y de código abierto ha sido identificado como una estrategia clave para democratizar la química computacional [5, 6].

Las brechas de accesibilidad no son exclusivamente tecnológicas, sino también pe-

dagógicas. La formación en química computacional y en inteligencia artificial requiere competencias interdisciplinarias que integran química, matemáticas, estadística y ciencias de la computación. La literatura en educación química destaca que la falta de programas formativos estructurados y de capacitación docente limita la incorporación efectiva de estas metodologías en el aula [1, 3]. La irrupción de la inteligencia artificial, particularmente del aprendizaje automático y los modelos generativos, ofrece oportunidades sin precedentes para reducir barreras de entrada mediante interfaces más intuitivas y flujos de trabajo automatizados. No obstante, también plantea nuevos riesgos de exclusión asociados al acceso desigual a datos de calidad, a recursos computacionales para entrenamiento de modelos y a la alfabetización en IA. Estudios recientes subrayan la necesidad de integrar principios de equidad, transparencia y sostenibilidad en el diseño de herramientas de IA para la química [4, 1].

En el marco de este curso, reconocer estas brechas resulta fundamental para adoptar un enfoque crítico e inclusivo. Se priorizará el uso de herramientas abiertas, reproducibles y accesibles, así como el desarrollo de competencias que permitan a los estudiantes evaluar de manera informada tanto el potencial como las limitaciones de la química computacional y la inteligencia artificial en distintos contextos.

La **Exploración *in silico*** constituye un punto de partida fundamental para comprender cómo la química computacional y la inteligencia artificial permiten estudiar sistemas químicos sin recurrir, al menos en una primera etapa, a experimentación directa en el laboratorio. El término “*in silico*” se utiliza por analogía con las expresiones *in vivo* e *in vitro*. Mientras que *in vivo* hace referencia a estudios realizados en organismos vivos y *in vitro* a experimentos llevados a cabo en sistemas controlados fuera de ellos, *in silico* alude a experimentos, simulaciones y análisis realizados mediante computadoras. El nombre proviene del **silicio**, elemento base de los semiconductores utilizados en los microprocesadores, y refleja el uso de modelos matemáticos, algoritmos y datos digitales para explorar propiedades moleculares, reactividad, interacciones y comportamiento dinámico de sistemas químicos.

Una pieza central de la exploración *in silico* es la representación molecular, ya que la forma en que se visualiza una molécula influye directamente en su interpretación química. Por ejemplo, consideremos a la molécula “3-hydroxypropanal”, de acuerdo con su nombre IUPAC, entre las representaciones más comunes se encuentra la bidimensional, con los hidrógenos apolares implícitos (Figura 1.1)

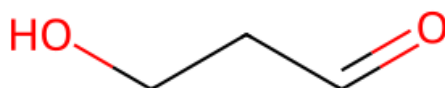


Figura 2.1: Representación bidimensional de 3-hydroxypropanal.

O el modelo **stick**, en el cual los enlaces se muestran como líneas o cilindros delgados

y los átomos suelen representarse de forma implícita o con pequeños nodos.

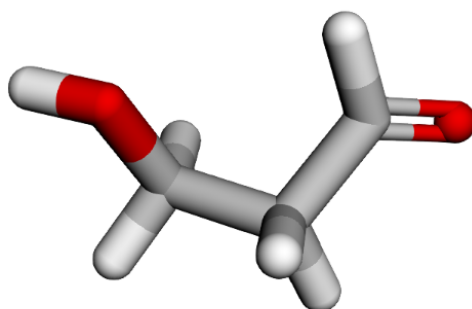


Figura 2.2: Representación tridimensional para 3-hydroxypropanal en modo de varillas o sticks.

Este tipo de representación es especialmente útil para analizar conectividad, geometría y conformación molecular. El modelo **ball-and-stick** incorpora esferas para los átomos y cilindros para los enlaces, facilitando la identificación de elementos químicos, ángulos de enlace y estereoquímica. Por ejemplo, en la Figura 1.3, las esferas etiquetadas como 1 y 5 corresponden a átomos de oxígeno, 2, 3 y 4, con átomos de carbono y todas las demás son átomos de hidrógeno.

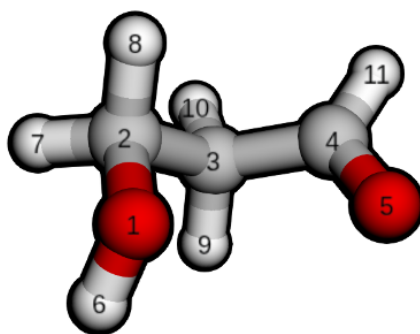


Figura 2.3: Representación 3D para 3-hydroxypropanal en modo de esferas y varillas.

Por otro lado, el modelo cartoon es ampliamente utilizado en bioquímica y biología estructural, particularmente para proteínas y ácidos nucleicos, donde se resaltan elementos de estructura secundaria como hélices alfa, láminas beta y la organización global del biomacromolécula, más que la posición exacta de cada átomo. Como es el caso de la molécula de la portada.

Para trabajar con estas representaciones, existen diversos programas de visualización molecular en tres dimensiones que se han consolidado como estándares de facto en investigación y docencia. PyMOL es uno de los más utilizados para la visualización de proteínas, complejos proteína-ligando y estructuras cristalográficas, destacando por su calidad gráfica y flexibilidad. VMD (Visual Molecular Dynamics) es ampliamente empleado en simulaciones de dinámica molecular, permitiendo el análisis de trayectorias y grandes sistemas biomoleculares. UCSF Chimera y su versión más reciente, ChimeraX, combinan visualización avanzada con herramientas de análisis estructural. Para moléculas pequeñas y quimiinformática, Avogadro y Jmol ofrecen entornos accesibles, multiplataforma y adecuados para enseñanza, mientras que programas como GaussView actúan como interfaces gráficas para paquetes de cálculo cuántico.

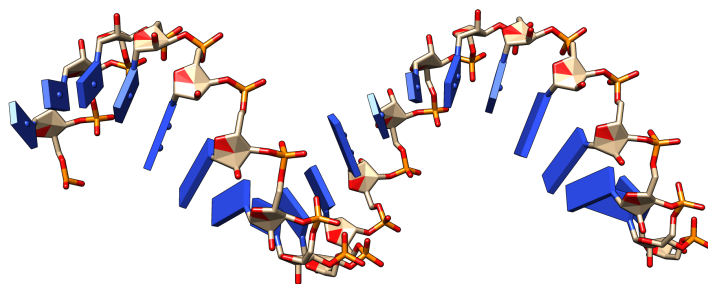


Figura 2.4: Representación cartoon de fragmento de ADN.

La exploración *in silico* se apoya, además, en el uso de formatos digitales de archivos moleculares, los cuales definen cómo se almacena y transmite la información estructural. Entre los formatos más básicos se encuentra el XYZ, que contiene el número de átomos seguido de una lista con el símbolo químico y las coordenadas cartesianas de cada átomo. Este formato es simple y ampliamente utilizado en química computacional, especialmente como salida o entrada en cálculos cuánticos. Sin embargo, el formato XYZ no incluye información explícita sobre enlaces, cargas formales ni tipos de átomos, lo que limita su uso para ciertos análisis.

```

1 11
2 Generated by RDKit
3 C -0.8002 0.3024 0.6968
4 O -2.0905 -0.0785 0.2395
5 C 0.2632 -0.2620 -0.2229
6 C 1.6139 0.2333 0.2027
7 O 2.5087 -0.5094 0.5938
8 H -0.7608 1.3966 0.7161
9 H -0.6708 -0.0508 1.7255
10 H -2.1696 -1.0433 0.3428
11 H 0.2477 -1.3570 -0.2060
12 H 0.0877 0.0450 -1.2593
13 H 1.7708 1.3238 0.1377
14

```

Figura 2.5: 3-hydroxypropanal escrita en formato XYZ.

El formato MOL, desarrollado originalmente dentro del ecosistema MDL, incorpora información más rica sobre la molécula, incluyendo conectividad, tipos de enlace y, en algunos casos, propiedades adicionales.

Es un formato muy utilizado en quimioinformática y bases de datos químicas, ya que permite reconstruir la estructura molecular de manera inequívoca. Cuando se requiere almacenar múltiples moléculas en un solo archivo, el formato SDF (Structure Data File) se convierte en una extensión natural del MOL, ya que permite incluir varias estructuras junto con campos de datos asociados, como propiedades fisicoquímicas, identificadores o resultados experimentales y computacionales. Por esta razón, SDF es ampliamente empleado en bibliotecas virtuales, cribado molecular y aprendizaje automático aplicado a química.

En el ámbito de la bioquímica estructural, el formato PDB (Protein Data Bank) ocupa un lugar central. Este formato fue diseñado para describir estructuras tridimensionales de macromoléculas biológicas, como proteínas y ácidos nucleicos, e incluye información

detallada sobre coordenadas atómicas, tipos de residuos, cadenas, ocupación, factores de temperatura y, en algunos casos, conectividad. Aunque presenta ciertas limitaciones históricas, como el manejo de sistemas muy grandes o de nuevos tipos de átomos, el formato PDB sigue siendo esencial para el estudio de interacciones biomoleculares, docking y simulaciones de dinámica molecular.

Existen otros formatos complementarios, como CIF para cristalografía, SMILES e InChI para representaciones lineales y compactas de moléculas, y formatos específicos de programas de simulación. Por ejemplo, para 3-hydroxypropanal el código SMILES es C(O)CC=O. En conjunto, el conocimiento de estos formatos y de sus alcances es un componente clave de la alfabetización digital en química, ya que permite al estudiante moverse con solvencia entre visualización, modelado, simulación e inteligencia artificial dentro del ecosistema de la química computacional moderna.

```

1
2      RDKit      3D
3
4  11 10  0  0  0  0  0  0  0  0  0999
5    -0.9024  -0.2213  0.1714 C
6    -0.7905   0.2491  1.5098 O
7     0.4747  -0.5767  -0.3517 C
8     1.2598   0.6829  -0.5875 C
9     1.7140   0.9936  -1.6853 O
10    -1.5423  -1.1092   0.1771 H
11    -1.3777   0.5577  -0.4335 H
12    -1.6950   0.4129   1.8324 H
13     0.3952  -1.1210  -1.2979 H
14     1.0315  -1.1864   0.3669 H
15     1.4327   1.3184   0.2983 H
16  1  2  1  0
17  1  3  1  0
18  3  4  1  0
19  4  5  2  0
20  1  6  1  0
21  1  7  1  0
22  2  8  1  0
23  3  9  1  0
24  3 10  1  0
25  4 11  1  0
26 M  END

```

Figura 2.6: 3-hydroxypropanal escrita en formato MOL.

2.1. Parte Práctica

2.1.1. Nivel básico: Conceptos básicos de Python

Ejercicio 1. Uso del comando print

El comando *print* permite mostrar en pantalla un mensaje de texto. El contenido mostrado no se almacena en memoria, se imprime como salida estándar.

```
1 # 1 Saludo
2 print("Hola mundo")
```

¿Cómo almacenas en memoria la salida de este código?

Ejercicio 2. Identificación del tipo de dato

La función *type* permite identificar el tipo de dato de un objeto. En este caso, se evalúa un número entero.

```
1 # Preguntar el tipo de dato
2 print(type(5))
```

Si solo ejecuto `type(5)` ¿Qué diferencias encuentras?

```
1 type(5)
```

Ejercicio 3. Tipo de dato string

Evalúa el tipo de dato correspondiente a:

```
1 type("a")
2 type("1")
3 type(" ")
```

¿Qué diferencias identificas?

Ejercicio 3. Operaciones basicas con cadenas de texto

Las cadenas de texto permiten realizar operaciones como concatenacion, repeticion y obtencion de su longitud, lo cual resulta fundamental para el procesamiento de informacion en Python.

Analiza el resultado de las siguientes instrucciones y explica que operacion se realiza en cada caso:

```
1 cadena = "Quimica"
2
3 print(cadena + " Digital")
4 print(cadena * 3)
5 print(len(cadena))
```

Describe que sucede en cada linea y discute por que estas operaciones no son validas para otros tipos de datos como los números enteros o reales.

```
1 type(1)
```

Ejercicio 4. Uso de la función help

La función *help* permite consultar palabras reservadas del lenguaje Python, mostrando documentación básica. ¿Cuál es la salida del siguiente código?

```
1 help("keywords")
```

Ejercicio 5. Asignación de variables

En el siguiente código se asigna una cadena de texto a una variable y luego se imprime su contenido.

```
1 mensaje = "Hay alguien en ese lugar?"  
2 print(mensaje)
```

¿Por qué **mensaje** no lleva comillas cuando es el argumento de **print**?

Ejercicio 6. Sensibilidad a mayúsculas y minúsculas

Python distingue entre variables con letras mayúsculas y minúsculas, tratándolas como identificadores diferentes.

```
1 c_color = 'blanco'  
2 c_Color = 'azul'  
3  
4 print(cat_color)  
5 print(cat_Color)
```

¿Qué diferencias encuentras?

Ejercicio 7. Operador módulo

El operador % devuelve el residuo de una división entera.

```
1 16 % 3
```

¿Cuál es la salida del código adjunto?

Ejercicio 8. División entera

El operador // realiza una división entera, descartando la parte decimal.

```
1 18 // 5
```

¿Cuál es la salida del código adjunto?

Ejercicio 9. Operador mayor o igual

Se evalúa una expresión lógica utilizando el operador mayor o igual que.

```
1 12 >= 3
```

¿Cuál es la salida del código adjunto? ¿Qué tipo de datos es?

Ejercicio 10. Operador distinto

El operador `!=` permite verificar si dos valores son diferentes.

```
1 12 != 12
```

¿Cuál es la salida del código adjunto? ¿Qué tipo de datos es?

Ejercicio 11. Operador de igualdad

El operador `==` evalúa si dos valores son iguales.

```
1 12 == 12
```

¿Cuál es la salida del código adjunto? ¿Qué tipo de datos es?

Ejercicio 12. Guardado de una cadena 'larga'

```
1 mensaje = "Las listas son una herramienta poderosa en Python"
2 mensaje[:]
```

¿Cuál es la salida del código adjunto? ¿Qué tipo de datos es?

Ejercicio 13. Longitud de una cadena

La función `len` permite obtener la cantidad de caracteres de una cadena.

```
1 len(mensaje)
```

¿Cuál es la longitud de `mensaje[:5]`?

Ejercicio 14. Acceso a un carácter específico

Se accede a un carácter de la cadena utilizando indexación.

```
1 mensaje[5]
```

Imprime el elemento 3 de la cadena `mensaje`.

Ejercicio 15. Acceso al último carácter

El índice negativo permite acceder al último carácter de la cadena.

```
1 mensaje[-1]
```

Imprime el penúltimo elemento de la cadena.

Ejercicio 16. Segmentación de cadenas

Extrae una subcadena especificando un rango de índices.

```
1 mensaje[3:10]
```

¿Cuál es la salida del código adjunto?

Ejercicio 17. Segmentación sin índice inicial

¿Cuál es la salida del siguiente código?

```
1 mensaje[:11]
```


Ejercicio 18. Segmentación hasta el final

¿Cuál es la salida del siguiente código?

```
1 mensaje[21:]
```

Ejercicio 19. Segmentación intermedia

Obtén una subcadena usando índices intermedios.

```
1 mensaje[77:113]
```

Ejercicio 20. Concatenación de subcadenas

Concatena dos fragmentos de una cadena para formar un nuevo texto.

```
1 # "Las listas son una herramienta poderosa"  
2 mensaje[:11] + mensaje[21:49]
```

Ejercicio 21. Repetición de cadenas

Repite una subcadena utilizando el operador de multiplicación.

```
1 # Componer: 'PythonPythonPythonPython'  
2 4 * mensaje[14:20]
```

Ejercicio 22. Concatenación con solapamiento

Concatena dos subcadenas parcialmente superpuestas.

```
1 # Solucion  
2 mensaje[3:15] + mensaje[5:40]
```

Ejercicio 23. Procesamiento de números telefónicos

Los teléfonos de una empresa tienen el siguiente formato prefijo-número-extensión donde el prefijo es el código del país +34, y la extensión tiene dos dígitos (por ejemplo +34-913724710-56). Escribir un programa que pregunte por un número de teléfono con este formato y muestre por pantalla el número de teléfono sin el prefijo y la extensión.

```
1 # Solucion  
2 minumero = input('Escribe tu numero: ')  
3  
4 guion = minumero.find("-")  
5 minumero[guion+1:-3]
```

Ejercicio 24. Conversión de dólares a soles y manipulación de decimales

En este ejercicio se solicita al usuario ingresar el precio de un producto en dólares con dos decimales. Posteriormente, el valor se convierte a soles utilizando un tipo de cambio fijo. Finalmente, el usuario indica la posición del punto decimal para separar soles y céntimos en la salida.

```

1  precio_dolares = input("Introduce el precio del producto con
    dos decimales: ")
2  precio_dolares = float(precio_dolares)
3
4  # Tipo de cambio fijo
5  tipo_cambio = 3.67
6  precio_soles = precio_dolares * tipo_cambio
7
8  precio_str = str(precio_soles)
9  print("En soles", precio_str)
10
11  posicion = int(input('Indicar posicion del punto: '))
12
13  soles = precio_str[:posicion]
14  centimos = precio_str[posicion+1:]
15
16  print(soles, "soles con", centimos, "centimos")

```

Ejercicio 25. Conversión de dólares a soles usando find()

Este ejercicio repite el proceso anterior, pero emplea la función *find* para localizar automáticamente la posición del punto decimal en la cadena.

```

1  precio_dolares = input("Introduce el precio del producto con
    dos decimales: ")
2  precio_dolares = float(precio_dolares)
3
4  tipo_cambio = 3.67
5  precio_soles = precio_dolares * tipo_cambio
6
7  precio_str = str(precio_soles)
8  print("En soles", precio_str)
9
10  punto = precio_str.find(".")
11
12  soles = precio_str[:punto]
13  centimos = precio_str[punto+1:]
14
15  print(soles, "soles con", centimos, "centimos")

```

Ejercicio 26. Intercambio de vocales en un texto

El programa solicita al usuario un texto y reemplaza todas las vocales a por o y las vocales e por i.

```

1  texto = input("Introduce un texto: ")
2
3  texto = texto.replace("a", "o")
4  texto = texto.replace("e", "i")
5
6  print(texto)

```

Ejercicio 27. Cambio de formato de fecha

Se pide al usuario ingresar una fecha en formato dd/mm/aaaa y se muestra en pantalla con el formato mm/dd/aaaa.

```
1  fecha = input("Introduce una fecha (dd/mm/aaaa): ")
2
3  dia = fecha[:2]
4  mes = fecha[3:5]
5  anio = fecha[6:]
6
7  nueva_fecha = mes + "/" + dia + "/" + anio
8  print(nueva_fecha)
```

Ejercicio 28. Separación de palabras en líneas independientes

El programa recibe una frase por consola y muestra cada palabra en una línea distinta.

```
1  frase = "Python no le importa si usted usa comillas simples o
2         dobles"
3
4  palabras = frase.replace(" ", "\n")
5
6  print(palabras)
```

También podemos emplear FOR para resolverlo:

```
1  frase = "Python no le importa si usted usa comillas simples o
2         dobles"
3
4  palabras = frase.split()
5
6  for palabra in palabras:
7      print(palabra)
```

2.2. Bucles: FOR

El bucle for en Python permite iterar sobre una secuencia de elementos, como listas, cadenas de texto o rangos de valores, ejecutando un bloque de código para cada elemento.

Ejercicio 29. Recorrido de una secuencia numérica

Este programa imprime los números del 1 al 5 utilizando la función range.

```
1  for i in range(1, 6):
2      print(i)
```

Ejercicio 30. Impresión de números pares

Se muestran únicamente los números pares comprendidos entre 0 y 10.

```
1  for i in range(0, 11, 2):
2      print(i)
```

Ejercicio 31. Recorrido de una lista

El bucle for se utiliza para recorrer una lista de elementos e imprimir cada uno de ellos.

```
1 elementos = ["H", "C", "N", "O"]
2
3 for elemento in elementos:
4     print(elemento)
```

Ejercicio 32. Conteo de caracteres en una cadena

Este ejercicio recorre una cadena de texto e imprime cada carácter en una línea diferente.

```
1 texto = "Python"
2
3 for letra in texto:
4     print(letra)
```

Ejercicio 33. Suma acumulativa

Se calcula la suma de los números del 1 al 10 usando un acumulador.

```
1 suma = 0
2
3 for i in range(1, 11):
4     suma = suma + i
5
6     print(suma)
```

Ejercicio 34. Tabla de multiplicar

El programa imprime la tabla de multiplicar del número 5.

```
1 numero = 5
2
3 for i in range(1, 11):
4     print(numero, "x", i, "=", numero * i)
```

Ejercicio 35. Conteo de vocales

Se cuentan las vocales presentes en una palabra ingresada por el usuario.

```
1 palabra = input("Introduce una palabra: ")
2 contador = 0
3
4 for letra in palabra:
5     if letra in "aeiou":
6         contador = contador + 1
7
8     print("Numero de vocales:", contador)
```

Ejercicio 36. Conversión de temperaturas

El bucle convierte una lista de temperaturas de grados Celsius a Kelvin.

```
1     temperaturas_c = [0, 20, 37, 100]
2     temperaturas_k = []
3
4     for t in temperaturas_c:
5         temperaturas_k.append(t + 273.15)
6
7     print(temperaturas_k)
```

Ejercicio 37. Generación de cuadrados

Se generan los cuadrados de los Generación del 1 al 5.

```
1     for i in range(1, 6):
2         print(i**2)
```

Ejercicio 38. Separación de palabras en líneas independientes

El programa recorre una frase y muestra cada palabra en una línea distinta.

```
1     frase = "La química digital integra programación y ciencia"
2
3     palabras = frase.split()
4
5     for palabra in palabras:
6         print(palabra)
```

2.3. Bucles: WHILE

El bucle `while` ejecuta un bloque de código mientras una condición lógica se mantenga verdadera. Es especialmente útil cuando no se conoce de antemano el número de iteraciones.

Ejercicio 39. Conteo simple

El programa imprime los números del 1 al 5 utilizando un bucle `while`.

```
1     i = 1
2
3     while i <= 5:
4         print(i)
5         i = i + 1
```

Ejercicio 40. Conteo de números pares

Se muestran los Generación pares comprendidos entre 0 y 10.

```
1     i = 0
2
3     while i <= 10:
```

```
4     print(i)
5     i = i + 2
```

Ejercicio 41. Suma acumulativa

Se calcula la suma de los números del 1 al 10 usando un acumulador.

```
1     i = 1
2     suma = 0
3
4     while i <= 10:
5         suma = suma + i
6         i = i + 1
7
8     print(suma)
```

Ejercicio 42. Tabla de multiplicar

El programa imprime la tabla de multiplicar del numero 6.

```
1     numero = 6
2     i = 1
3
4     while i <= 10:
5         print(numero, "x", i, "=", numero * i)
6         i = i + 1
```

Ejercicio 43. Recorrido de una cadena

Este ejercicio imprime cada caracter de una cadena de texto en una linea distinta.

```
1     texto = "Python"
2     i = 0
3
4     while i < len(texto):
5         print(texto[i])
6         i = i + 1
```

Ejercicio 44. Conteo de vocales

Se cuentan las vocales presentes en una palabra ingresada por el usuario.

```
1     palabra = input("Introduce una palabra: ")
2     i = 0
3     contador = 0
4
5     while i < len(palabra):
6         if palabra[i] in "aeiou":
7             contador = contador + 1
8             i = i + 1
9
10    print("Numero de vocales:", contador)
```

Ejercicio 45. Control de entrada

El programa solicita Generación al usuario hasta que ingrese el valor cero.

```
1  numero = int(input("Introduce un numero: "))
2
3  while numero != 0:
4  numero = int(input("Introduce un numero: "))
```

Ejercicio 46. División repetida

Se divide un numero entre 2 hasta que el resultado sea menor que 1.

```
1  numero = float(input("Introduce un numero: "))
2
3  while numero >= 1:
4  numero = numero / 2
5  print(numero)
```

Ejercicio 47. Validación de contraseña

El programa solicita una contraseña hasta que el usuario introduzca la correcta.

```
1  contrasena = "python"
2  entrada = input("Introduce la contrasena: ")
3
4  while entrada != contrasena:
5  entrada = input("Introduce la contrasena: ")
6
7  print("Acceso permitido")
```

Ejercicio 48. Contador regresivo

Se imprime una cuenta regresiva desde 5 hasta 1.

```
1  i = 5
2
3  while i >= 1:
4  print(i)
5  i = i - 1
```

2.4. Condicionales: IF

Las estructuras condicionales permiten ejecutar bloques de código solo si se cumple una condición lógica. En Python, la instrucción `if` se utiliza para la toma de decisiones.

Ejercicio 49. Número positivo o negativo

El programa evalúa si un numero ingresado por el usuario es positivo, negativo o cero.

```
1  numero = float(input("Introduce un numero: "))
2
3  if numero > 0:
```

```

4     print("El numero es positivo")
5
6     if numero < 0:
7         print("El numero es negativo")
8
9     if numero == 0:
10        print("El numero es cero")

```

Ejercicio 50. Evaluación de mayoría de edad

Se determina si una persona es mayor de edad a partir de su edad.

```

1     edad = int(input("Introduce tu edad: "))
2
3     if edad >= 18:
4         print("Es mayor de edad")
5
6     if edad < 18:
7         print("Es menor de edad")

```

Ejercicio 51. Numero par

El programa verifica si un numero entero es par.

```

1     numero = int(input("Introduce un numero entero: "))
2
3     if numero % 2 == 0:
4         print("El numero es par")

```

Ejercicio 52. Comparación de dos números

Se comparan dos Generación y se indica cual es mayor.

```

1     a = float(input("Introduce el primer numero: "))
2     b = float(input("Introduce el segundo numero: "))
3
4     if a > b:
5         print("El primer numero es mayor")
6
7     if b > a:
8         print("El segundo numero es mayor")

```

Ejercicio 53. Aprobación de un curso

El programa indica si un estudiante aprueba un curso en funcion de su nota final.

```

1     nota = float(input("Introduce la nota final: "))
2
3     if nota >= 11:
4         print("Curso aprobado")
5
6     if nota < 11:
7         print("Curso desaprobado")

```


Ejercicio 54. Evaluación de temperatura

Se evalúa si una temperatura es alta en función de un valor umbral.

```
1 temperatura = float(input("Introduce la temperatura: "))
2
3 if temperatura > 37:
4     print("Temperatura elevada")
```

Ejercicio 55. Verificación de vocal

El programa determina si un carácter ingresado es una vocal.

```
1 letra = input("Introduce una letra: ")
2
3 if letra in "aeiou":
4     print("La letra es una vocal")
```

Ejercicio 56. Evaluación de múltiples condiciones

Se determina si un número está dentro de un intervalo específico.

```
1 numero = float(input("Introduce un número: "))
2
3 if numero >= 0 and numero <= 10:
4     print("El número está en el intervalo [0,10]")
```

Ejercicio 57. Comparación de cadenas

El programa compara dos cadenas de texto.

```
1 texto1 = input("Introduce el primer texto: ")
2 texto2 = input("Introduce el segundo texto: ")
3
4 if texto1 == texto2:
5     print("Los textos son iguales")
```

Ejercicio 58. Verificación de acceso

Se valida el acceso a un sistema mediante una clave predefinida.

```
1 clave = "python123"
2 entrada = input("Introduce la clave: ")
3
4 if entrada == clave:
5     print("Acceso permitido")
```

2.5. List Comprehension

La comprensión de listas permite crear nuevas listas de forma compacta y legible a partir de secuencias existentes, aplicando operaciones y condiciones en una sola línea.

Ejercicio 59. Generación de una lista de números

Genera una lista con los números del 1 al 10 utilizando comprensión de listas.

```
1 numeros = [i for i in range(1, 11)]  
2 print(numeros)
```

Ejercicio 60. Cuadrados de números

Genera una lista con los cuadrados de los números del 1 al 5.

```
1 cuadrados = [i**2 for i in range(1, 6)]  
2 print(cuadrados)
```

Ejercicio 61. Números pares

Construye una lista que contiene solo los números pares entre 0 y 10.

```
1 pares = [i for i in range(11) if i % 2 == 0]  
2 print(pares)
```

Ejercicio 62. Conversión de temperaturas

Convierte una lista de temperaturas de grados Celsius a Kelvin.

```
1 temperaturas_c = [0, 20, 37, 100]  
2 temperaturas_k = [t + 273.15 for t in temperaturas_c]  
3 print(temperaturas_k)
```

Ejercicio 63. Longitud de palabras

Obtener una lista con la longitud de cada palabra en una frase.

```
1 frase = "la quimica digital integra ciencia y programacion"  
2 longitudes = [len(palabra) for palabra in frase.split()]  
3 print(longitudes)
```

Ejercicio 64. Filtrado de números positivos

Construye una lista que contiene solo los números positivos.

```
1 numeros = [-3, -1, 0, 2, 5, 8]  
2 positivos = [n for n in numeros if n > 0]  
3 print(positivos)
```

Ejercicio 65. Conversión de cadenas a mayúsculas

Convierte una lista de palabras a letras mayúsculas.

```
1 palabras = ["python", "quimica", "datos"]  
2 mayusculas = [p.upper() for p in palabras]  
3 print(mayusculas)
```

Ejercicio 66. Extracción de vocales

Extraer las vocales presentes en una palabra.

```
1 palabra = "programacion"
2 vocales = [letra for letra in palabra if letra in "aeiou"]
3 print(vocales)
```

Ejercicio 67. Valores mayores a un umbral

Filtrar los valores mayores a un umbral dado.

```
1 datos = [3, 7, 2, 9, 4, 10]
2 filtrados = [x for x in datos if x > 5]
3 print(filtrados)
```

Ejercicio 68. Creación de tuplas

Genera una lista de tuplas con números y sus cuadrados.

```
1 pares_cuadrados = [(i, i**2) for i in range(1, 6)]
2 print(pares_cuadrados)
```

2.6. Funciones

Las funciones permiten organizar el código en bloques reutilizables, facilitando la lectura, el mantenimiento y la modularidad de los programas. En Python, las funciones se definen usando la palabra reservada. `def`.

Ejercicio 69. Función sin parámetros

Se define una función que imprime un mensaje en pantalla cuando es llamada.

```
1 def saludo():
2     print("Hola desde una funcion")
3
4     saludo()
```

Ejercicio 70. Función con un parámetro

La función recibe un nombre como argumento y muestra un saludo personalizado.

```
1 def saludar(nombre):
2     print("Hola", nombre)
3
4     saludar("Ana")
```

Ejercicio 71. Función con dos parámetros

Se define una función que suma dos números ingresados como parametros.

```
1 def suma(a, b):  
2     resultado = a + b  
3     return resultado  
4  
5     print(suma(3, 5))
```

Ejercicio 72. Función que retorna un valor

La función calcula el cuadrado de un número y devuelve el resultado.

```
1 def cuadrado(x):  
2     return x**2  
3  
4     print(cuadrado(4))
```

Ejercicio 73. Función para determinar si un numero es par

Se define una funcion que evalua si un numero entero es par.

```
1 def es_par(numero):  
2     if numero % 2 == 0:  
3         return True  
4     else:  
5         return False  
6  
7     print(es_par(6))
```

Ejercicio 74. Función con lista como argumento

La función recibe una lista de números y devuelve la suma total.

```
1 def suma_lista(numeros):  
2     total = 0  
3     for n in numeros:  
4         total = total + n  
5     return total  
6  
7     print(suma_lista([1, 2, 3, 4]))
```

Ejercicio 75. Función para calcular promedio

Se calcula el promedio de una lista de valores numéricos.

```
1 def promedio(valores):  
2     total = 0  
3     for v in valores:  
4         total = total + v  
5     return total / len(valores)  
6  
7     print(promedio([10, 15, 20]))
```

Ejercicio 76. Función con valor por defecto

La función utiliza un parámetro con valor por defecto.

```
1 def potencia(base, exponente=2):
2     return base**exponente
3
4 print(potencia(3))
5 print(potencia(3, 3))
```

Ejercicio 77. Función que procesa cadenas

Se define una función que cuenta la cantidad de vocales en una cadena.

```
1 def contar_vocales(texto):
2     contador = 0
3     for letra in texto:
4         if letra in "aeiou":
5             contador = contador + 1
6     return contador
7
8 print(contar_vocales("programacion"))
```

Ejercicio 78. Función que devuelve múltiples valores

La función devuelve dos valores: el mínimo y el máximo de una lista.

```
1 def min_max(lista):
2     minimo = min(lista)
3     maximo = max(lista)
4     return minimo, maximo
5
6 print(min_max([3, 7, 1, 9]))
```

2.7. Manejo de archivos

El manejo de archivos permite leer, escribir y procesar información almacenada en disco, lo cual es esencial para trabajar con datos experimentales, resultados de simulaciones y registros computacionales.

Ejercicio 79. Creación de un archivo de texto

Se crea un archivo de texto y se escribe una línea de información en él.

```
1 archivo = open("datos.txt", "w")
2 archivo.write("Introduccion a la quimica digital\n")
3 archivo.close()
```

Ejercicio 80. Escritura de múltiples líneas

Se escriben varias líneas en un archivo de texto usando un bucle.

```
1     archivo = open("lineas.txt", "w")
2
3     for i in range(1, 6):
4         archivo.write(f"Linea {i}\n")
5
6     archivo.close()
```

Ejercicio 81. Lectura completa de un archivo

Se lee el contenido completo de un archivo de texto.

```
1     archivo = open("datos.txt", "r")
2     contenido = archivo.read()
3     archivo.close()
4
5     print(contenido)
```

Ejercicio 82. Lectura línea por línea

Se imprime el contenido de un archivo leyendo línea por línea.

```
1     archivo = open("lineas.txt", "r")
2
3     for linea in archivo:
4         print(linea)
5
6     archivo.close()
```

Ejercicio 83. Conteo de líneas en un archivo

Se cuenta el numero de líneas contenidas en un archivo.

```
1     archivo = open("lineas.txt", "r")
2     lineas = archivo.readlines()
3     archivo.close()
4
5     print(len(lineas))
```

Ejercicio 84. Uso del gestor de contexto with

Se utiliza la instrucción with para manejar archivos de forma segura.

```
1     with open("datos.txt", "r") as archivo:
2         contenido = archivo.read()
3
4     print(contenido)
```

Ejercicio 85. Escritura y lectura combinadas

Se escribe información en un archivo y luego se lee su contenido.

```
1 with open("resultado.txt", "w") as archivo:
2     archivo.write("Simulacion finalizada\n")
3
4 with open("resultado.txt", "r") as archivo:
5     print(archivo.read())
```

Ejercicio 86. Almacenamiento de números en un archivo

Se guardan números en un archivo de texto y luego se leen.

```
1 with open("numeros.txt", "w") as archivo:
2     for i in range(1, 6):
3         archivo.write(str(i) + "\n")
4
5 with open("numeros.txt", "r") as archivo:
6     print(archivo.read())
```

Ejercicio 87. Lectura de datos y conversión a enteros

Se convierten las líneas de un archivo a números enteros.

```
1 with open("numeros.txt", "r") as archivo:
2     numeros = [int(linea) for linea in archivo]
3
4     print(numeros)
```

Ejercicio 88. Copia del contenido de un archivo

Se copia el contenido de un archivo en otro archivo nuevo.

```
1 with open("datos.txt", "r") as origen:
2     contenido = origen.read()
3
4 with open("copia_datos.txt", "w") as destino:
5     destino.write(contenido)
```

2.8. Librerías

Las librerías en Python permiten extender las capacidades del lenguaje mediante módulos externos o personalizados. En esta sección se practican distintas formas de importación y uso de librerías ampliamente empleadas en ciencia de datos, visualización y computación científica.

Ejercicio 89. Importación de una librería propia

Se asume la existencia de un archivo llamado `milibreria.py` en el mismo directorio de trabajo.

```
1 import milibreria
2
3 milibreria.saludar()
```

Nota: El archivo `milibreria.py` debe estar en el mismo directorio del cuaderno o script.

Ejercicio 90. Uso de la librería `math`

Se importa la librería `math` para calcular la raíz cuadrada y el valor de `pi`.

```
1 import math
2
3 print(math.sqrt(25))
4 print(math.pi)
```

Instalación: `math` viene incluida en Python, no requiere instalación.

Ejercicio 91. Importación selectiva desde `math`

Se importa solo la función `sin` desde la librería `math`.

```
1 from math import sin, pi
2
3 resultado = sin(pi / 2)
4 print(resultado)
```

Ejercicio 92. Uso básico de `numpy`

Se crea un arreglo numerico y se calcula su promedio.

```
1 import numpy as np
2
3 datos = np.array([1, 2, 3, 4, 5])
4 print(np.mean(datos))
```

Instalación:

- Jupyter local: `pip install numpy`
 - Google Colab: ya viene instalado
-

Ejercicio 93. Generación de datos aleatorios con `numpy`

Se generan números aleatorios con distribución uniforme.

```
1 import numpy as np
2
3 aleatorios = np.random.rand(5)
4 print(aleatorios)
```

Ejercicio 94. Graficación simple con matplotlib

Se grafica una función lineal.

```
1 import matplotlib.pyplot as plt
2
3 x = [0, 1, 2, 3]
4 y = [0, 2, 4, 6]
5
6 plt.plot(x, y)
7 plt.show()
```

Instalación:

- En Jupyter local: `pip install matplotlib`
 - En Google Colab: ya viene instalado
-

Ejercicio 95. Visualización estadística con seaborn

Se genera un grafico de distribución.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 datos = [1, 2, 2, 3, 3, 3, 4]
5 sns.histplot(datos)
6 plt.show()
```

Instalación:

- Jupyter local: `pip install seaborn`
 - Google Colab: ya viene instalado
-

Ejercicio 96. Análisis de valores faltantes con missingno

Se visualizan valores perdidos en un conjunto de datos.

```
1 import missingno as msno
2 import pandas as pd
3
4 df = pd.DataFrame({
5     "A": [1, None, 3],
6     "B": [4, 5, None]
7 })
8
9 msno.matrix(df)
```

Instalación:

- Jupyter local: `pip install missingno`
 - Google Colab: Ya viene instalado
-

Ejercicio 97. Uso básico de scikit-learn

Se crea un conjunto de datos artificial para regresión.

```

1  from sklearn.datasets import make_regression
2
3  X, y = make_regression(n_samples=10, n_features=1)
4  print(X)
5  print(y)

```

Instalación:

- Jupyter local: `pip install scikit-learn`
- Google Colab: ya viene instalado

—

Ejercicio 98. Normalización de datos con sklearn

Se normalizan datos usando StandardScaler.

```

1  from sklearn.preprocessing import StandardScaler
2  import numpy as np
3
4  X = np.array([[1], [2], [3]])
5  scaler = StandardScaler()
6  X_norm = scaler.fit_transform(X)
7
8  print(X_norm)

```

—

Ejercicio 99. Importación de RDKit

Se verifica la importación de RDKit para química computacional.

```

1  from rdkit import Chem
2
3  mol = Chem.MolFromSmiles("CCO")
4  print(mol)

```

Instalación:

- Jupyter local (conda recomendado): `conda install -c conda-forge rdkit`
- Google Colab: `!pip install rdkit-pypi`

—

Ejercicio 100. Combinación de múltiples librerías

Se combinan numpy y matplotlib para graficar una función matemática.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.linspace(0, 2 * np.pi, 100)
5  y = np.sin(x)
6
7  plt.plot(x, y)
8  plt.show()
```

En la primera parte hemos visto exclusivamente ejercicios de Python, ahora vemos cómo implementarlos con el modelamiento molecular.

2.8.1. Nivel Intermedio

Ejercicio 101. Implementación en modelamiento molecular

“Optimiza” 10 moléculas con actividad farmacológica

2.8.2. Nivel Avanzado

Ejercicio 102. Formación de metalofármacos

Emplea MolSimplify para formar complejos de hierro con las moléculas empleadas en el ejercicio anterior.